Lecture-17 [29 September]

Recap: Monte-Carlo, TD Learning, and Q-Learning Methods for the Newsvendor Problem

Objective: Determine $a^* \in \underset{a \in \{0,\dots,a_{max}\}}{\arg\max}\ q(a)$,

where $q(a) := \mathbb{E}[\text{Revenue}(a, Z)]$

$$= \sum_{z=0}^{a-1} [Gz - L(a-z)]\ \mathbb{P}(Z=z) + Ga\ \mathbb{P}(Z \geq a),$$

where $G = g - c$, $L = c - h$, $c \to$ cost price per unit, $g \to$ selling price per unit, $h \to$ scrap value per unit

If the distribution of Z is known, then

$$a^* = F_Z^{-1}\left(\frac{G}{G+L}\right),$$ (rounded appropriately to make a^* an integer)

where F_Z is the CDF of Z, and

$$F_Z^{-1}(u) := \inf\{z \in \mathbb{R} : F_Z(z) \geq u\}, \quad u \in (0,1).$$

When F_Z is unknown, we may resort to simulation.

a) Monte-Carlo Simulation:

- Obtain IID samples $z_1, z_2, \dots, z_N$ from the underlying distribution F_Z
- Set

$$\hat{q}_N(a) := \frac{1}{N} \sum_{t=1}^{N} \text{Revenue}(a, z_t), \quad a \in \{0, \dots, a_{max}\}.$$

- Determine

$$a_N^* \in \underset{a}{\arg\max}\ \hat{q}_N(a).$$

$\hat{q}_N(a) \xrightarrow{a.s.} q(a) \quad \forall a \in \{0,\dots,a_{max}\}$

b) Temporal differencing:

- Start with $q_1(a) = 0 \quad \forall a$.
- Use the update

$\forall a, \forall n \geq 1$:

$$q_{n+1}(a) = (1-\tau_n)\, q_n(a) + \tau_n\, \text{Revenue}(a, z_n)$$

$$= q_n(a) + \tau_n\, (\text{Revenue}(a, z_n) - q_n(a)),$$

Where $\{\tau_n\}_{n\geq 1}$ is a sequence of positive constants converging to 0.

τ_n is the learning rate at iteration n.

Robbins-Monro conditions

Remarks:

① $\{\tau_n\}_{n\geq 1}$ is usually chosen so that $\sum_{n=1}^{\infty} \tau_n^2 < +\infty$, $\sum_{n=1}^{\infty} \tau_n = +\infty$.

(Eg: $\tau_n = 1/n$)

② The above TD approach is referred to as TD(0).

c) Q-learning and SARSA

Here, we use the recursion

Q-learning: $$q_{n+1}(a) = q_n(a) + \tau_n \left(\max_{a'} \text{Revenue}(a', z_n) - q_n(a) \right), \quad n \geq 1.$$

Alternatively, one may sample $a' \sim \text{Unif}\{0, \dots, a_{max}\}$ and use

SARSA: $$q_{n+1}(a) = q_n(a) + \tau_n \left(\text{Revenue}(a', z_n) - q_n(a) \right), \quad n \geq 1.$$

$\max_{a'}$ → greedy policy (exploitation), $a' \sim$ Unif → exploration

A strategy with a mix of exploration and exploitation would be:

Pick $$a' = \begin{cases} a^* \in \arg\max_a \text{Revenue}(a, z_n), & \text{w.p. } 1-\varepsilon_n, \\ a \in \{0, \dots, a_{max}\} \setminus \{a^*\}, & \text{w.p. } \frac{\varepsilon_n}{a_{max}}, \end{cases}$$ (ε_n-greedy sampling)

and set $$q_{n+1}(a) = q_n(a) + \tau_n \left(\text{Revenue}(a', z_n) - q_n(a) \right).$$

Another alternative to sample a' is using softmax rule

$$p_n(a') = \frac{e^{\eta_n q_n(a')}}{\sum_a e^{\eta_n q_n(a)}},$$

and setting $$q_{n+1}(a) = q_n(a) + \tau_n \left(\text{Revenue}(a', z_n) - q_n(a) \right).$$

Q-Learning: An "Earning While Learning" Perspective

- Set $q_1(a) = 0 \quad \forall a$, $R_0 = 0$.
- Choose a_1 greedily, ε-greedily, or by softmax sampling

- Implement a_1 and observe $r_1 = \text{Revenue}(a_1, z_1)$
- Set $R_1 = R_0 + r_1$
- Update $q_2(a_1)$ using one of Q-learning or SARSA update rules

⋮

- Repeat for a total of N iterations

- At the end, we have R_N and $q_N = [q_N(a) : a \in \{0, \ldots, a_{max}\}]$.
- Finally, pick $a_N^* \in \underset{a}{\text{argmax}}\, q_N(a)$.

By tuning η_n, τ_n and based on update rule, different variants of Q-learning can be compared on the basis of running average of cumulative reward & on how accurately they determine a^*.

10.2.2 A numerical study

This section illustrates and discusses computation using the above approaches.

Model

Demand is generated from either a discrete (rounded) and truncated normal distribution with mean 50 and standard deviation 15 or a gamma distribution[17] with shape parameter 20 and scale parameter 4. The item cost is 20, the salvage value is 5, and price varies among 21, 30, and 100. The corresponding G values are 1, 10, and 80, and $L = 15$. The corresponding ratios of $G/(G+L)$ are 0.0625, 0.400, and 0.842, illustrating several distinct quantiles of the demand distribution. Set $A_0 = \{0, 1, \ldots, 120\}$.

Monte Carlo estimation

Monte Carlo methods are compared using sample sizes of $N = 100$ and $N = 5{,}000$. A replicate consists of estimating $q(a)$ for each $a \in A_0$ for each price. The expected reward is estimated using (10.5) and the optimal order quantity is chosen according to (10.6). Using a common random number seed, the same realized demand was used in each replicate. There was no need to resample; estimates could be based on stored demand samples.

Table 10.2 gives the optimal order quantity (for a typical replicate) using the closed form representation (10.3) and that based on the arg max of the Monte Carlo estimates of $q(a)$. Of course, choosing $N = 5{,}000$ produces more accurate estimates of the optimal order quantity than using $N = 100$ but the difference was surprisingly small. An exception was when demand was normal and price was 21, when the optimal order quantity corresponded to $G/(G+L) = 0.0625$. This is because larger sample sizes are needed to estimate values in the tail of a distribution.

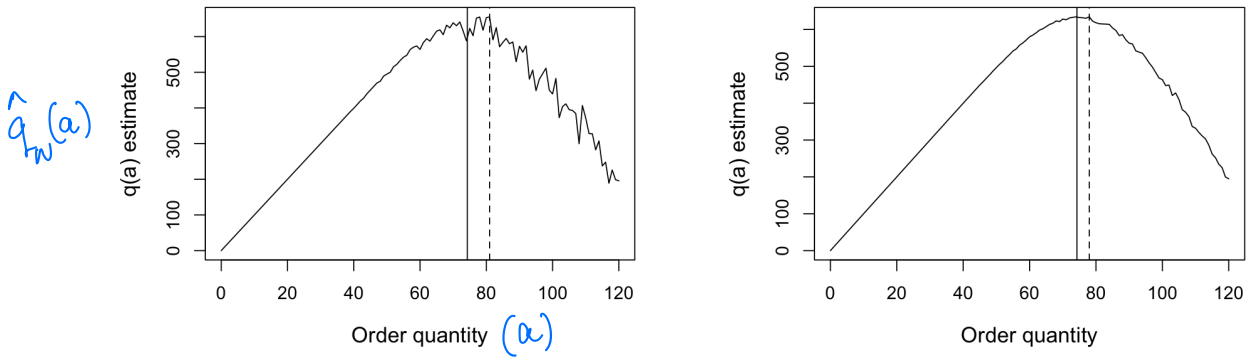
Distribution	Price	Optimal	Monte Carlo ($N = 100$)	Monte Carlo ($N = 5{,}000$)
Normal	21	27	36	26
	30	46	42	44
	100	65	69	64
Gamma	21	55	57	55
	30	74	81	78
	100	98	109	104

Table 10.2: Optimal order quantity and that based on Monte Carlo estimates of $q(a)$ in a typical replicate of the newsvendor model using $N = 100$ and $N = 5{,}000$.

Figure 10.2 shows $\hat{q}(a)$ based on a single replicate with gamma distributed demand, price equal to 30 and two values of N. Observe that the estimates with $N = 100$ are

[17] For this choice of parameters, the mean of the gamma distribution is 80 and its standard deviation is 17.89.

considerably noisier than those with $N = 5{,}000$. However, surprisingly, the estimated optimal order quantities vary little. Note that for both values of N the curves are quite similar for small values of a. Moreover, Figure 10.2b shows that $\hat{q}(a)$ is quite flat near the true optimum, suggesting that there may be considerable variability in choosing the action using Monte Carlo estimates.



(a) $N = 100$ (b) $N = 5{,}000$

Figure 10.2: Estimates of $q(a)$ based on a single replicate with gamma distributed demand and price equal to 30 for two sample sizes. The solid vertical line represents the theoretical optimal order quantity and the dashed line that obtained using (10.6).

Temporal differencing

This section illustrates the use of temporal differencing by applying it to estimate $q(80)$ for $G = 80$, $L = 15$ and gamma distributed demand. It compares learning rates $\tau_n = 0.05, n^{-1}, 0.5n^{-0.75}$, and $10/(40 + n)$. Figure 10.3 shows the evolution of the iterates of (10.7) over a typical replicate with $N = 1{,}000$ iterations starting from 0 and the same demand sequence for each learning rate. Observe that the sample mean, which corresponds to $\tau_n = n^{-1}$, best approximates the true value, with the estimate using $\tau_n = 0.5n^{-0.75}$ quite close. Note also that the estimate corresponding to $\tau_n = 0.05$ oscillates. More formally, estimates can be compared using the RMSE as described in Appendix 10.11.1. The respective RMSE values corresponding to the above learning rates are 467.21, 163.87, 214.29 and 419.08, consistent with Figure 10.3. Such calculations over multiple configurations and with many replicates provide the basis for parameter studies in this chapter and the next.

Q-learning

Applying Q-learning using (10.9) was more challenging than the methods above. The ϵ-greedy action selection method was unreliable. Even when ϵ_n was slightly less than one,

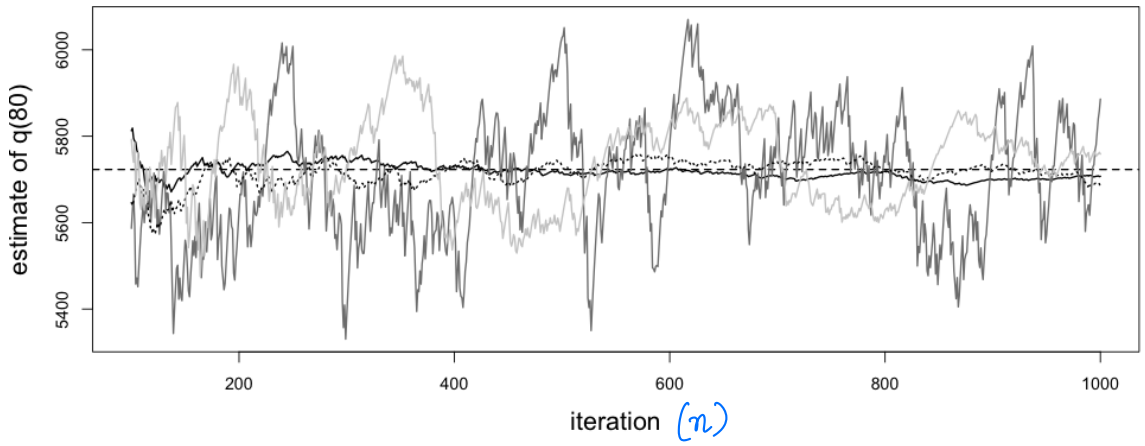


Figure 10.3: Comparison, based on a single replicate, for estimates of $q(80)$ for the four choices of τ_n described in the text. The horizontal dashed line represents the exact value $q(80) = 5{,}723$, and the other four lines represent different choices of τ_n. The dark grey line corresponds to $\tau_n = 0.05$, the solid line to $\tau_n = n^{-1}$, the dotted line to $\tau_n = 0.5/(n^{0.75})$ and the light grey line to $\tau_n = 10/(n + 40)$.

a single high-value action was sampled too frequently; all other actions were sampled equally and infrequently. Thus the recursion "got stuck" at a potentially non-optimal action.

Softmax action selection was better suited to this problem since there are 121 actions and $q(a)$ was quite flat near the optimum and dropped off quickly as shown in Figure 10.2b. Consequently, softmax sampled high-value actions more frequently, thus providing more accurate estimates of $q(a)$ close to the optimum.

After preliminary experimentation, three learning rates, $\tau_n = 5/(20 + n)$, $\tau_n = 0.5n^{-0.5}$ and $\tau_n = 0.05/(1 + (10^{-5})n^2)$, were chosen for further analyses. The third choice for τ_n is justified later in this chapter. Note that estimates based on $\tau_n = n^{-1}$ and $\tau_n = 0.5n^{-0.75}$ converged too quickly, producing very inaccurate estimates of $q(a)$ and the optimal order quantity.

This study used the scenarios in the model description above and generated 40 replicates for each using common random number seeds for each replicate. For each scenario, the recursions began with $q(a) = 0$, set $N = 50{,}000$ and the softmax parameter $\eta_n = 1/6{,}000$ for all n. The number of times an action was previously chosen (as opposed to the iteration number) was used as the index of the learning rate.

Table 10.3 summarizes results in terms of the mean and standard deviation of the optimal order quantify over 40 replicates for each configuration of distribution, price and learning rate. The learning rate $\tau_n = 0.5n^{-0.5}$ best estimated the optimal order quantity on average. However, the estimates were more variable when the price equaled 100. Note that when $N = 10{,}000$, $\tau_n = 0.5n^{-0.5}$ produced the most accurate estimates

but their variability was greater.

Distribution	Price	Optimal	$\tau_n = 5/(20+n)$	$\tau_n = 0.05/\left(1+(10^{-5})n^2\right)$	$\tau_n = 0.5n^{-0.5}$
Normal	21	27	29.79 (2.89)	30.72 (3.97)	28.59 (2.73)
	30	46	48.64 (3.09)	48.41 (4.00)	46.62 (2.92)
	100	65	67.69 (5.91)	68.74 (6.70)	67.18 (8.01)
Gamma	21	55	57.62 (3.05)	57.15 (3.66)	55.18 (2.60)
	30	74	76.41 (3.91)	76.64 (4.22)	74.64 (4.54)
	120	98	99.87 (6.32)	104.13 (6.71)	99.15 (8.65)

Table 10.3: Optimal order quantity in the newsvendor model and mean (standard deviation) of estimates based on 40 replicates of Q-learning with three different learning rates.

Q-learning: An "Earning while learning" perspective

The above analysis is from the perspective of solving an optimization problem; that is, Q-learning is an online method to optimize the order quantity. Alternatively, from a simultaneous earning and learning perspective, the newsvendor might seek to maximize revenue while learning what quantity to order.

To illustrate this approach, consider an instance with normal demand, price equal 30, $N = 100{,}000$ iterates, the three learning rates used above, and softmax action selection with $\eta_n = 600n^{0.2}$. The index for the learning rate was the number of times an action was previously used and the index for softmax sampling was the iteration number. The effect of this specification for η_n is to allow random action choice at the beginning and then make it more likely that sampling is done from actions with high values of the estimate of $q(a)$, which is shown in Figure 10.4. Observe that in this replicate, the estimate of $q(a)$ is smoothest around the optimal order quantity where most of the actions are chosen.

Figure 10.5 shows the running average revenue in a typical replicate[18]. Observe that all three learning rates result in learning as indicated by increasing average revenue per iterate. Using the learning rate $\tau_n = 5/(20+n)$ resulted in the greatest revenue early on. Eventually, revenue streams from all three learning rates become approximately the same but that using $\tau_n = 5/(20+n)$ continued to have the greatest average revenue. Note that over 100,000 iterates even a small increase in average revenue contributes to a large increase in total revenue.

10.2.3 Conclusions

The purpose of this section was to provide an introduction to the simulation methods in this chapter. By choosing a practical and easy to understand **one-state** example, several implementation issues that arise in multi-state models were swept under the rug.

[18] In 40 random sequences, the average revenues always exhibited this pattern.

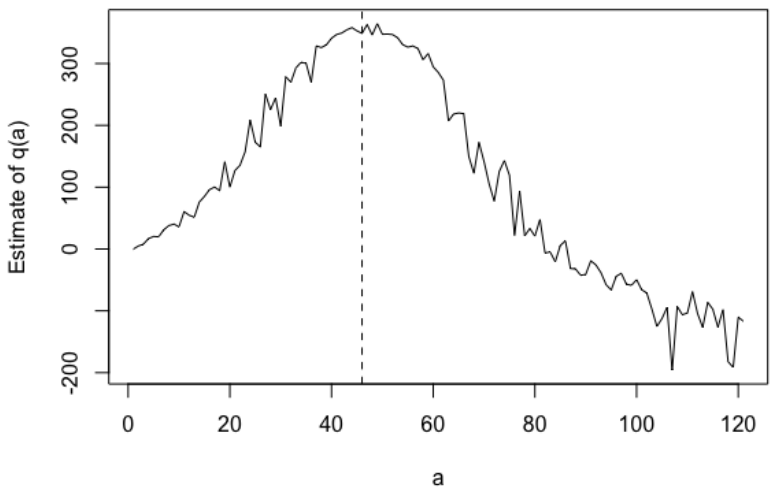


Figure 10.4: Estimate of $q(a)$ based on 100,000 iterates of Q-learning. The vertical dashed line corresponds to a^* found using (10.3).

However, this example was sufficiently rich to provide insight into issues arising when using these techniques. The remainder of this chapter generalizes these approaches to multi-state models.

Some noteworthy observations include:

1. Monte Carlo methods are easy to understand, however learning does not occur until all data has been collected. As the above calculations show, Monte Carlo methods involve costly experimentation for actions that may be far from optimal. However, because $q(a)$ is relatively flat near the maximum, large sample sizes near this value are useful. Note that Monte Carlo could be integrated with a search procedure to reduce experimentation.

2. Temporal differencing provides an online approach for policy value function estimation. This means that estimates are available after each observation. However, the quality of the approximation depends significantly on the learning rate. Note that $\tau_n = n^{-1}$ corresponds to an online version of Monte Carlo.

3. Q-learning worked best with softmax sampling and on-policy updates using (10.9). Again, outcomes were sensitive to the learning rates. After preliminary experimentation three promising learning rates were used in a more detailed study. Optimal order quantity estimates were quite similar with the exponential learning rate working slightly better than the others in determining an order quantity. From the perspective of learning, the ratio learning rate generated the greatest revenue. Differences in revenue over learning rates was greatest initially.

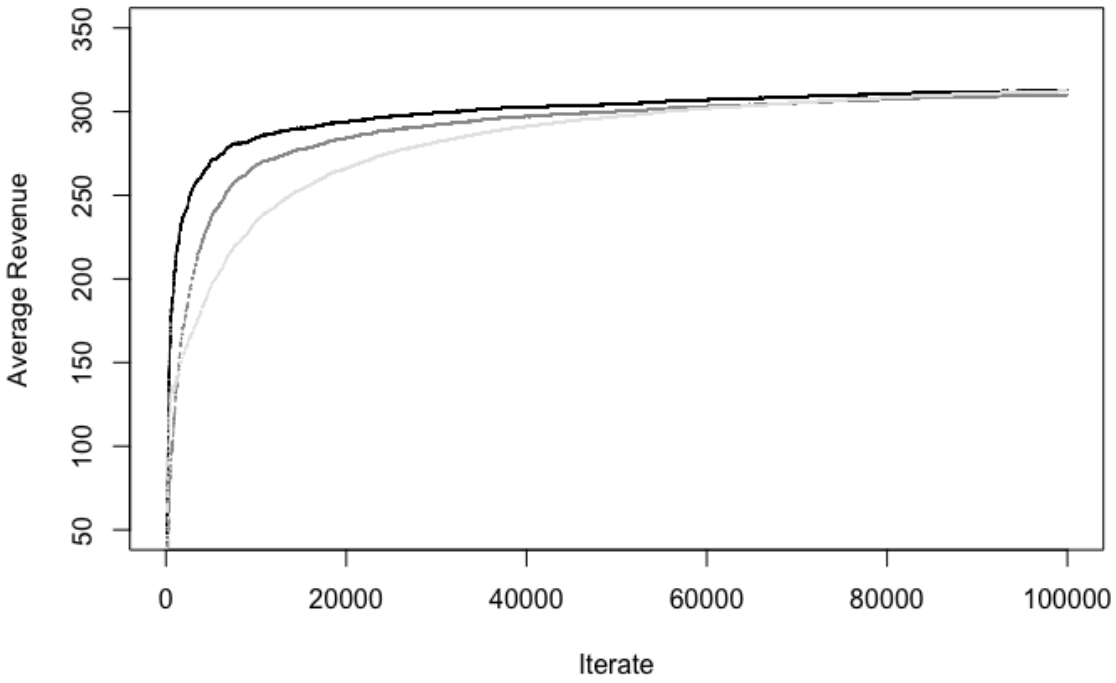


Figure 10.5: Cumulative average revenue vs. iterate using Q-learning. Data is from a single replicate of newsvendor model using softmax sampling and three learning rates. The black line corresponds to $\tau_n = 5/(20 + n)$, the dark grey line (between the other two) to $\tau_n = 0.05/(1 + (10^{-5})n^2)$ and the light grey to $\tau_n = 0.5n^{-0.5}$.

10.3 Policy evaluation: Episodic models

In an episodic model, the system accrues rewards until it reaches a reward-free absorbing state at some (possibly) random stopping time in the future. Usually this will be when a task is completed or a game ends. This section focuses on models in which stopping times are determined by the evolution of the system. For example, in a robot navigation model, an episode terminates when the robot achieves its goal[19].

As noted in Section 2.3.2, a discounted model may be viewed as an episodic model in which the episode terminates at a geometrically distributed stopping time independent of the evolution of the system. Policy evaluation for discounted models will be discussed in Section 10.4 below.

Finite horizon models may also be regarded to be episodic models in which the stopping time is fixed and after which the system transitions to a zero-reward absorbing

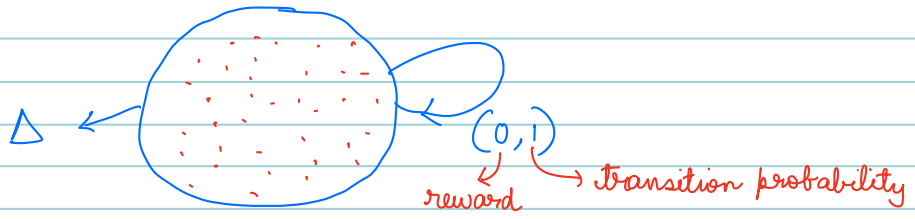
[19]The episode could end when the robot fails, but computation in various applications suggests that it is more expedient to terminate an episode only when the goal is achieved.

Policy evaluation — Episodic Models

Simulation for estimating v_∞^π in Infinite Horizon Total Reward Models

Setting:

* State space consists of a non-empty set of zero-reward absorbing states



* Given a policy $\pi \in \Pi^{HR}$, define

$$v_\infty^\pi(s) = E^\pi\left[\sum_{n=1}^{N} r(S_{n-1}, A_{n-1}, S_n) \mid S_0 = s\right], \quad s \in \mathcal{S}.$$

Here, N denotes the random time of absorption, i.e.,

$$N = \inf\{n \geq 1 : S_n \in \Delta\}.$$

* Fact: Optimal policies in this setting are stationary. Therefore, it suffices to fix our attention to Π^{SR}.

The Key Question:

Using simulation methods, how can we estimate v_∞^ϕ for a given $\phi : \mathcal{S} \to \Delta(\mathcal{A})$?

Monte-Carlo Methods for estimating v_∞^ϕ

(1) Starting-state Monte-Carlo method

Algorithm 10.1: Starting-state Monte-Carlo Policy Evaluation

1. Initialise:
   a) Specify $\phi : \mathcal{S} \to \Delta(\mathcal{A})$, number of episodes (replicates) K
   b) Specify $\bar{s} \in \mathcal{S} \setminus \Delta$ and create an empty list VALUES
   c) $k \leftarrow 1$
2. Iterate: While $k \leq K$:
   a) $s \leftarrow \bar{s}$ and $v \leftarrow 0$
   b) Generate an episode (replicate): While $s \notin \Delta$:

i) Sample $a \sim \phi(\cdot|s)$

ii) Simulate (s', R) (if P, r are unknown)

OR

Sample $s' \sim P(\cdot|s,a)$ and set $R \leftarrow r(s,a,s')$ (if P, r are known)

iii) Update

$$v \leftarrow v + R$$

iv) $s \leftarrow s'$.

c) Terminate episode ($s \in \Delta$)

i) Append v to VALUES

ii) $k \leftarrow k+1$

3. Terminate: Return

$$\hat{V}_\infty^\phi(\bar{s}) = \text{mean}(\text{VALUES})$$

Remarks:

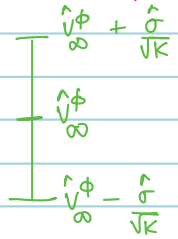
* Algorithm 10.1 generates an estimate of $V_\infty^\phi(\bar{s})$ for a given $\bar{s} \in \mathcal{S}$. It must be run for each $\bar{s} \in \mathcal{S}$ to obtain an estimate of V_∞^ϕ.

* If $\phi : \mathcal{S} \to \mathcal{A}$, then 2(b)(i) becomes "set $a = \phi(s)$".

* By SLLN, as the number of episodes $K \to \infty$,

$$\hat{V}_\infty^\phi(\bar{s}) \xrightarrow{K \to \infty} V_\infty^\phi(\bar{s}) \qquad \forall\, \bar{s} \in \mathcal{S}.$$

* By maintaining the list VALUES, one can obtain standard deviation ($\hat{\sigma}$) of the estimates & also compute the standard deviation error bars



(2) First-visit Monte-Carlo

Algorithm 10.2 [First-visit Monte-Carlo]

1. Initialise:

a) Specify $\phi : \mathcal{S} \to \Delta(\mathcal{A})$, number of episodes (replicates) K

b) For each $s \in \mathcal{S}$, create an empty list VALUES(s)

c) $k \leftarrow 1$

2. Iterate: While $k \leq K$:

a) Create an empty list VISITED

b) $v(s) \leftarrow 0$ for all $s \in \mathcal{S} \setminus \Delta$

c) Specify $s \in S \setminus \Delta$.

d) Generate an episode: While $s \notin \Delta$:

i) If $s \notin$ VISITED, append s to VISITED

ii) Sample $a \sim \phi(\cdot|s)$.

iii) Simulate (s', R) (if P, r are unknown)

OR

Sample $s' \sim P(\cdot|s,a)$ and set $R \leftarrow r(s,a,s')$ (if P, r are known)

iv) For all $s'' \in$ VISITED, set

$$v(s'') \leftarrow v(s'') + R.$$

$S = \{A, B, C, \dagger\}$

$A \xrightarrow{2} B \xrightarrow{-1} C \xrightarrow{1} A \xrightarrow{2} C \xrightarrow{3} \dagger$

$v(A) = 2 - 1 + 1 + 2 + 3$

$v(B) = -1 + 1 + 2 + 3$

$v(C) = 1 + 2 + 3$

v) $s \leftarrow s'$.

e) Terminate episode ($s \in \Delta$):

i) For each $s'' \in$ VISITED, append $v(s'')$ to VALUES(s'')

ii) $k \leftarrow k+1$.

3. Terminate: For each $s \in \mathcal{S} \setminus \Delta$, return [. . . . .]

$$\hat{v}_{\infty}^{\phi}(s) = \text{mean}(\text{VALUES}(s)).$$

Remarks:

* VALUES$_k(s'')$ stores the total reward accumulated starting from first visit to state s'' in episode k.

* Let $H_s = \inf\{n \geq 1 : S_n = s\}$ be the first hitting time of $s \in \mathcal{S}$.

Then, it turns out that

$$\mathbb{E}\left[\sum_{n=H_s+1}^{N} r(S_{n-1}, A_{n-1}, S_n) \,\middle|\, \{H_s < N\}\right] = v_{\infty}^{\phi}(s) \qquad \forall s \in \mathcal{S}.$$

* The above algorithm approximates the expectation on the LHS via

$$\hat{V}_{\infty}^{\phi}(s) = \frac{\sum_{k:\ \text{state } s \text{ is visited at least once in episode } k} \ \sum_{n=H_s+1}^{N} r(S_{n-1}, A_{n-1}, S_n)}{\#\text{ episodes in which } s \text{ is visited at least once}}$$